

We can easily see how the camera motion emulates an actual player. On triggering the *j_flag* using the key press, the Y axis value of the eye position and the scene centre increase, giving the illusion of jumping. It is here that we also increment the flag value used to count the number of consecutive jumps. This is needed to avoid multiple jumps in mid-air. On hitting the maximum, we call *reset()* to reset it, and this in turn triggers the *grav_flag*, calling the *grav()*. Once the player has returned to the lowest point possible, we stop the recurring call of *grav()* that was happening; instead, we reset the flag value to 0 so that the player can jump again.

It is in this code segment that we also use the inbuilt function, *lights()*, which gives the world we are viewing more depth of field by using shading and shadowing. You should now be able to jump and see an output similar to what's shown in Figures 3 and 4.

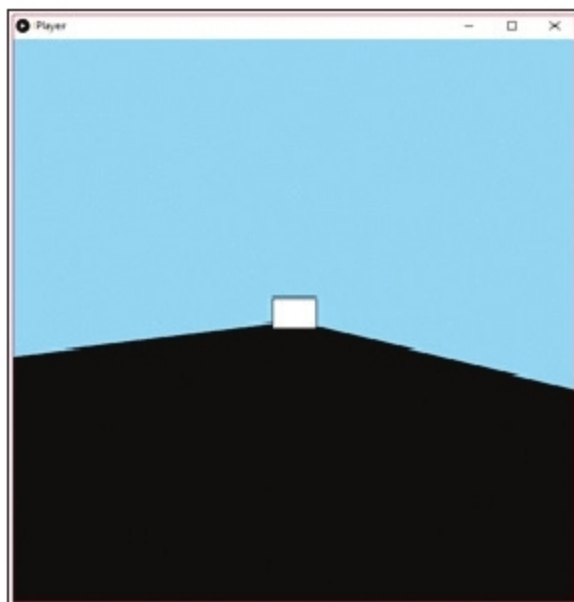


Figure 3: Path view

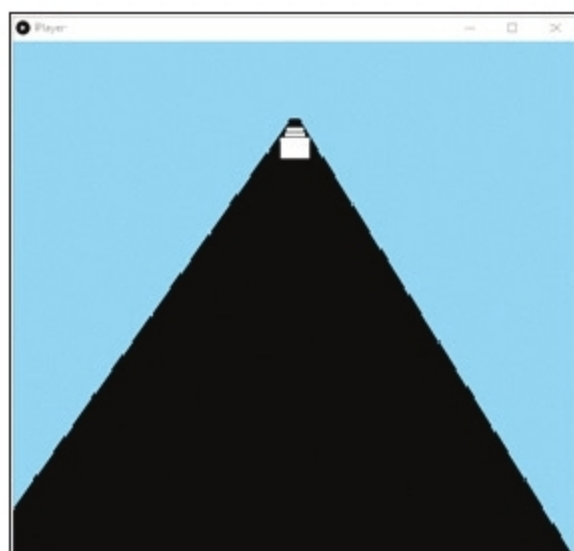


Figure 4: Jump view

Adding 'game over'

Compared to what we have already achieved in this tutorial, what comes next is quite easy. In this part, we will look at how to add the collision mechanism and also at how to add the replay functionality. We will achieve this in the same way as the previous game, by using a global variable to toggle an *if-else* block.

```
gameOver = False
```

Now, we have to check if the player hits any block during the game play. For this, after checking the bounds of the obstacle in the *for* loop, simply append the following *if* statement, as shown below:

```
for ob in obs:
    pushMatrix()
    translate(width/2, base_ht-
ob.ht, ob.z)
    fill(255)
    box(200, ob.ht, 50)
    popMatrix()
    ob.move()
    ob.check_bounds()

    if ob.z >= 55:
        if ((height/pl.eyeY) -
(base_ht-ob.ht)) > 0:
            gameOver = True
```

This *if* block runs if the block is five units away from the camera, in which case it again checks if the collision occurs based on the height of the camera position and the height of the object.

With this, you can create your game's *if-else* block to ensure that the game ends when a collision takes place.

```
if gameOver == False:
    #code covered till this point
else:
    background(0)
    ortho()
    textSize(32);
    textAlign(CENTER)
    fill(200);
    text("GAME OVER", width/2-2,
height/2-2)
    fill(255)
    text("GAME OVER", width/2,
```

```
height/2)
    textSize(14)
    text("Press Space To Replay",
width/2, 550)
```

As you can see, if *gameOver* becomes *True*, we paint the screen black like in the previous game. However, since we are rendering 3D, we need to set the drawing back to 2D before displaying any text, for which we use *ortho()*.

Note: Calling *perspective()* and *camera()* will set it back to the defaults, making the result the same as calling *ortho()*. However, only calling *ortho()* will make the drawing truly 2D.

Now, to add the replay functionality, we just add another condition to *keyPressed()*. We can assign the trigger to be the space bar, and when pressed, we simply reset the *gameOver* variable to be *False*.

```
def keyPressed():
    global pl, gameOver, obs
    #code till this point
    if key == ' ':
        gameOver = False
        pl.score = 0
        pl.speed = 40
        z_val = -500*10
        for ob in obs:
            ob.z = z_val
            z_val -= 850
```

By the end of this part, you should get a screen like the one shown in Figure 5 when you crash into an obstacle.



Figure 5: 'Game Over' screen